ensamblador (para programar) y el **programa ensamblador** (que lo traduce a lenguaje de máquina).

- ✓ Cada máquina o familia de procesadores tiene su propio juego de instrucciones — imposible intercambiar programas entre familias distintas.
- ✓ Distintas versiones de una misma CPU pueden tener juegos de instrucciones incompatibles, o incorporar instrucciones nuevas.

Solución: los **lenguajes de alto nivel**, que abstraen todo esto (ej.: escribir Z=X+Y en vez de instrucciones de máquina específicas de cada micro).

> ¿Cómo se ejecuta un programa de alto nivel sobre una máquina que solo entiende 0s y 1s? Con

> **programas traductores**: interpretación, compilación, o ambas combinadas. No es decisión del programador —que solo elige el lenguaje que le convenga— sino de quien crea el lenguaje.

Interpretación

- ✓ Un programa escrito en un lenguaje interpretado (ej.: Lisp, Smalltalk, Basic, Python, Ruby, PHP, Perl).
- ✓ Un programa llamado **Intérprete** lo traduce en el momento de la ejecución, repitiendo por cada sentencia: **leer → analizar → decodificar → ejecutar**.
- ✓ Solo pasa por ciertas instrucciones, no por todas (según la ejecución).
- ✓ Tiene, por cada acción posible, un subprograma en lenguaje de máquina que la ejecuta; interpretar es llamar a esos subprogramas en la secuencia adecuada.

Compilación

- ✓ Se elige un lenguaje de alto nivel de este tipo (ej.: Ada, C++, Fortran, Pascal).
- ✓ Un programa llamado **Compilador** traduce todo el programa a lenguaje de máquina **antes** de ejecutarlo.
- ✓ Pasa por todas las instrucciones antes de ejecutar; el código generado se guarda y se puede reusar ya compilado.
- ✓ Esquema: programa en **lenguaje fuente** → Programa Compilador → programa en **lenguaje objeto** (ejecutable, o nivel intermedio/ensamblador).

Comparación Intérprete vs. Compilador

CRITERIO	INTÉRPRETE	COMPILADOR
Cómo se ejecuta	Línea a línea; necesita siempre el intérprete instalado; el fuente será público.	Se usa antes de ejecutar; produce un programa equivalente en lenguaje objeto; el fuente no será público.
Orden de ejecución	Orden lógico — no necesariamente recorre todo el código.	Orden físico de las sentencias — recorre todo.
Tiempo de ejecución	Decodifica cada sentencia (repetitivo); en lazos repite tantas veces como haga falta.	Traduce todo una sola vez, no repite lazos; genera código ya compilado.
Eficiencia posterior	Más lento — repite el proceso en cada ejecución.	Más rápido — ya está en bajo nivel y compilado.
Espacio ocupado	Menos memoria — no recorre todo; tablas de símbolos más dinámicas.	Más espacio — una sentencia puede generar muchas instrucciones; tablas siempre generadas.
Detección de errores	Fácil — la sentencia fuente se relaciona directo con la que se ejecuta.	Difícil — se pierde la referencia entre fuente y objeto; hacen falta otras técnicas.

Combinación de técnicas

- ✓ Interpretación pura y compilación pura son dos extremos; en la práctica muchos lenguajes combinan ambas.
- ✓ **1) Primero interpreto y luego compilo** — el intérprete se usa en desarrollo para diagnosticar errores; con el programa validado, se compila para un código objeto más eficiente.
- ✓ **2) Primero compilo y luego interpreto** — se traduce a un código intermedio de bajo nivel que luego se interpreta; sirve para generar **código portable**. Ej.: Java compila a *bytecode*, que luego interpreta la JVM en la máquina cliente (Windows/Mac/Unix); también C#/.NET, VB.NET, Python.

COMPILADORES — CÓMO FUNCIONAN

Traducen todo el programa; pueden generar un código ejecutable (.exe) o un código intermedio (.obj). Ejemplos: C, Ada, Pascal. La compilación puede hacerse en 1 o 2 grandes etapas:

- ✓ **1) Etapa de Análisis** (ligada al código fuente): análisis léxico (Scanner) → análisis sintáctico (Parser) → análisis semántico (semántica estática).
- ✓ **2) Etapa de Síntesis** (ligada al código objeto, hardware y arquitectura): optimización del código → generación del código. Entre ambas etapas se puede generar código intermedio.

Análisis léxico (Scanner)

Es el que más tiempo lleva; analiza a nivel de palabra (**lexema**). Divide el programa en identificadores, delimitadores, símbolos especiales, números, palabras clave/reservadas, comentarios; valida cada token, filtra comentarios y blancos, y da error si algo no matchea ninguna categoría léxica. Carga los identificadores en la **tabla de símbolos**. Resultado: el descubrimiento de los tokens.

```
x:=a+b*c;
// tokens → (id,"x") (op,":=") (id,"a") (op,"+") (id,"b") (op,"*") (id,"c") (punt,";"
```

Análisis sintáctico (Parser)

Analiza a nivel de sentencia; identifica estructuras (sentencias, declaraciones, expresiones) apoyándose en los tokens, alternándose con el análisis léxico y semántico. Usa una gramática para construir el **árbol sintáctico** del programa y ver si lo que entra es correcto.

Análisis semántico (semántica estática)

Fase medular: procesa las estructuras que reconoció el sintáctico y sigue armando la estructura del código ejecutable, agregando información implícita (ej.: variables no declaradas). Aplica **gramática de atributos** para comprobar tipos, agrega descriptores a la tabla de símbolos, y comprueba duplicados, tipos y nombres. Es el **nexo** entre la etapa de Análisis y la de Síntesis.

Generación de código intermedio

Transforma el código fuente en una representación para una **máquina abstracta**, independiente de la máquina real (a diferencia del código objeto, que sí depende de ella). Debe ser fácil de producir y de traducir al programa objeto. El más habitual: **código de 3 direcciones** — toda sentencia se pasa a proposiciones `x := y op z`, con condicionales simples y saltos.

```
while (a>0) and (b<(a*4-5)) do a:=b*a-10;

L1: if (a>0) goto L2          L4: t1:=b*a
    goto L3                       t2:=t1-10
L2: t1:=a*4                       a:=t2
    t2:=t1-5                      goto L1
    if (b < t2) goto L4       L3: ...
    goto L3
```

Etapa de Síntesis

- ✓ Construye el programa ejecutable y genera el código necesario.
- ✓ Interviene el **Linkeditor**: si hubo traducción separada de módulos, enlaza los distintos módulos objeto (unidades, librerías, funciones, macros...) y genera el módulo de carga (programa objeto completo).
- ✓ Se puede optimizar (optativo) — no lo hacen todos los compiladores; puede ser una herramienta independiente o venir incluida.
- ✓ El **Loader** lo carga en memoria para ejecutarse.

Qué se optimiza: velocidad vs. tamaño del ejecutable, código para un microprocesador específico, comprobación de rangos/pila, expresiones booleanas, código o funciones no usadas, etc. Ej.: si hay 2 saltos (`goto`) seguidos, se pueden reducir a 1 solo.

> **Esquema completo:** Programa fuente → Analizador Léxico → Analizador Sintáctico → Analizador Semántico (estática) → Generador de código intermedio → Optimizador de código intermedio → Generador de código objeto → Optimizador de código → Programa objeto → Linkeditor / Loader → Computadora (E/S).

Próxima clase: **Semántica Operacional** — ligadura, descriptores, momentos de ligadura y estabilidad; variables, arquitectura Von Neumann, atributos; nombre, alcance, tipo, L-valor, R-valor, alias.

3

Semántica Operacional — Entidades

Clase 3

Con qué "cosas" trabaja un programa (entidades), cómo se les liga la información que necesitan (binding), y el desglose completo de la variable en la tupla <Nombre, Alcance, Tipo, L-value, R-value>.

ENTIDADES, ATRIBUTOS Y LIGADURA (BINDING)

Los programas trabajan con **entidades**; las entidades tienen **atributos**; esos atributos tienen que establecerse antes de poder usar la entidad. La **ligadura (binding)** es la asociación entre la entidad y el atributo. El **descriptor** es el lugar donde se almacenan los atributos.

- ✓ **Variable** → atributos: nombre, tipo, área de memoria, etc.
- ✓ **Rutina** → atributos: nombre, parámetros formales, parámetros reales, etc.
- ✓ **Sentencia** → atributo: acción asociada.

Los lenguajes se diferencian en: el número de entidades, el número de atributos que se les pueden ligar, el **momento** en que se hacen las ligaduras (binding time), y la **estabilidad** de la ligadura (una vez establecida, ¿se puede modificar?).

MOMENTO Y ESTABILIDAD DE LA LIGADURA

Momentos posibles, de más estático a más dinámico: **Definición del lenguaje** → **Implementación del lenguaje** → **Compilación** → **Ejecución.** Los tres primeros son "estáticos"; el último es "dinámico".

- ✓ **Ligadura estática:** se establece antes de la ejecución y no se puede cambiar. "Estático" refiere tanto al momento del binding como a su estabilidad.
- ✓ **Ligadura dinámica:** se establece en tiempo de ejecución y puede cambiar según alguna regla del lenguaje. Excepción: las constantes.

Ejemplos por momento (en C, con la variable `int a`)

- ✓ **En Definición:** forma de las sentencias, estructura del programa, nombres de tipos predefinidos.
- ✓ **En Implementación:** representación de los números y sus operaciones.
- ✓ **En Compilación:** se liga el tipo a la variable (`int a` → a es entera).
- ✓ **En Ejecución:** la variable se liga a su valor y a su lugar de almacenamiento — el valor puede cambiar muchas veces.

> Estático vs. dinámico en general: las reglas dinámicas son más fáciles de implementar, pero son menos claras para la disciplina de programación y hacen el código más difícil de leer.

LA VARIABLE COMO ABSTRACCIÓN

La variable es una abstracción: **variable/nombre/sentencia de asignación ↔ celda de memoria/dirección/modificación destructiva del valor**.

```
x = 8
// ¿qué me dispara esa sentencia? ¿qué información me da, y cuál?
```

Esa pregunta es la que arma la tupla que describe a toda variable:
<Nombre, Alcance, Tipo, L-value, R-value>

- ✓ **Nombre:** string de caracteres usado para referenciar la variable (identificador).
- ✓ **Alcance:** rango de instrucciones en el que se conoce el nombre.
- ✓ **Tipo:** conjunto de valores + conjunto de operaciones.
- ✓ **L-value:** lugar de memoria asociado con la variable (tiempo de vida).
- ✓ **R-value:** valor codificado almacenado en la ubicación de la variable.

NOMBRE — ASPECTOS DE DISEÑO

- ✓ **Longitud máxima:** Fortran 6 caracteres; Python sin límite; C depende del compilador (suele ser 32, se ignora el resto); Pascal/Java/ADA cualquier longitud.
- ✓ **Caracteres aceptados (conectores):** Python/C/Pascal usan _; Ruby usa solo minúsculas para variables locales y $ para empezar nombres de variables globales.
- ✓ **Sensibilidad a mayúsculas/minúsculas:** C y Python son sensibles (`Sum ≠ sum ≠ SUM`); Pascal no lo es.
- ✓ (Ya visto en Clase 1: palabra reservada vs. palabra clave.)

ALCANCE

El alcance de una variable es el rango de instrucciones en el que se conoce su nombre (**visibilidad**). Las instrucciones pueden manipular una variable a través de su nombre dentro de su alcance. Cada lenguaje adopta reglas propias para ligar un nombre a su alcance.
Alcance estático (léxico)

- ✓ Define el alcance según la estructura léxica (textual) del programa.
- ✓ Se liga a una declaración examinando el texto del programa, **sin necesidad de ejecutarlo**.
- ✓ Es el que adopta la mayoría de los lenguajes.

Alcance dinámico

- ✓ Define el alcance en términos de la **ejecución**: cada declaración extiende su efecto a todas las instrucciones ejecutadas después, hasta encontrar una nueva declaración con el mismo nombre durante la ejecución.
- ✓ Lenguajes: APL, Lisp (original), Afnix (Aleph), Tcl, Perl.

```
int x;                  // x externa
{ /*bloque A*/ int x; }
{ /*bloque B*/ int x; }
{ /*bloque C*/ x = ...; }  // C no declara x
```

Si C es invocado desde A o desde B: con **alcance dinámico**, la x usada es la de quien invocó a C (la de A, o la de B, según el caso) — menos legible. Con **alcance estático**, en ambos casos se referencia siempre a la x externa (la más cercana en el texto), sin importar quién invocó.

Esto se comprobó igual en C, Pascal, ADA y Python: aun con procedimientos/funciones anidados, la

variable resuelta es siempre la más cercana **en el texto** (alcance léxico), nunca la del bloque que hizo el llamado — en ADA incluso una constante que se define usando una variable externa toma el valor que esa variable tenía en ese punto del texto, no un valor redeclarado después.

```
x = 200
def uno():
    x = 10
    def dos():
        global x       // liga la asignación a la x GLOBAL (200)
        x = x + 1
    def tres():
        nonlocal x     // liga la asignación a la x del ámbito contenedor (la de uno,
        x = x + 1
```

En Python, sin `global`/`nonlocal`, toda asignación dentro de una función crea siempre una variable **local nueva** (por eso hacen falta esas palabras clave para modificar una variable de un ámbito exterior).

Conceptos asociados al alcance

- ✓ **Local:** referencias creadas dentro del programa o subprograma.
- ✓ **No local:** referencias usadas dentro del subprograma, pero no creadas en él.
- ✓ **Global:** referencias creadas en el programa principal.

Espacios de nombres (namespaces)

Una zona separada donde se pueden declarar y definir objetos, funciones y cualquier identificador (tipo, clase, estructura, etc.) con nombre propio. Sirven para evitar conflictos entre identificadores con el mismo nombre en proyectos grandes o al usar bibliotecas externas.

TIPO

Un tipo es un **conjunto de valores** + un **conjunto de operaciones**. Antes de que una variable pueda ser referenciada debe ligársele un tipo. El tipo protege a las variables de operaciones no permitidas; el **chequeo de tipos** verifica su uso correcto.

- ✓ **Predefinidos** (tipos base): descriptos en la definición del lenguaje. Ej.: `boolean` — valores true/false, operaciones and/or/not; sus valores se ligan en la implementación a una representación de máquina.
- ✓ **Definidos por el usuario:** el programador declara tipos nuevos a partir de los predefinidos y constructores. Ej. Pascal: `type t = array[1..10] of integer;` liga el nombre `t` con "arreglo de 10 enteros" en compilación; `t` hereda todas las operaciones del arreglo.
- ✓ **TADs** (Tipos de Datos Abstractos): no hay ligadura por defecto — el programador especifica la representación y las operaciones. TAD = estructura de datos que representa al nuevo tipo + rutinas para manipular sus objetos (ej.: una clase Pila en C++, con su estructura interna y sus operaciones push/pop/etc.).

Momento del tipo — Estático

Se liga en compilación y no puede cambiar; el chequeo también es estático. La ligadura puede ser:

- ✓ **Explícita:** mediante una declaración (`int x, y; bool z;`).
- ✓ **Inferida:** el tipo de una expresión se deduce de sus componentes (típico de lenguajes funcionales, ej. Lisp).
- ✓ **Implícita:** se deduce por reglas del lenguaje (ej. Fortran: variables de la I a la N son enteras por defecto, el resto reales).

Lenguajes con tipo estático: Fortran, Pascal, Algol, Simula, ADA, C, C++, Java, etc.

Momento del tipo — Dinámico

Momento del tipo — Dinámico

Se liga en ejecución y puede cambiar. Más flexible (programación genérica), pero más costoso —hay que mantener descriptores en runtime—, con chequeo dinámico y menor legibilidad. Lenguajes: APL, Snobol, JavaScript, Python, Ruby, etc.

L-VALUE

- ✓ Es el **área de memoria** ligada a la variable.
- ✓ **Tiempo de vida** (lifetime): período de tiempo que existe la ligadura.
- ✓ **Alocación:** momento en que se reserva la memoria. El tiempo de vida es el tiempo en que la variable está alocada.

Momentos de alocación

- ✓ **Estática:** sensible a la historia — persiste durante toda la ejecución del programa.
- ✓ **Dinámica — Automática:** cuando aparece la declaración. **Explícita:** a través de algún constructor (ej. new).
- ✓ **Persistente:** su tiempo de vida no depende de la ejecución — existe en el ambiente (ej.: archivos, bases de datos).

Dos invocaciones distintas de un mismo procedimiento generan dos alocaciones diferentes para su misma variable local. Un procedimiento recursivo genera una alocación nueva de su variable local por **cada** invocación.

R-VALUE

Es el valor almacenado en el l-valor de la variable; se interpreta según su tipo. Un objeto queda definido por el par **(l-valor, r-valor)**.

```
x := x + 1
// la "x" de la izquierda aporta su l-valor (dónde escribir)
// la "x" de la derecha aporta su r-valor (qué leer)
```

Se accede a la variable a través del l-valor; se puede modificar el r-value. Es **dinámico por naturaleza**: b:=a copia el r-valor de a al l-valor de b.
Constantes: se "congela" el valor (`const pi = 3.1416`). En Pascal la ligadura de la constante es estática; en Ada es dinámica-estable.
Inicialización — ¿cuál es el r-valor de una variable recién creada?

- ✓ **Ignorar el problema:** queda lo que hubiera en esa memoria (basura).
- ✓ **Por defecto:** ej. enteros en 0, caracteres en blanco.
- ✓ **En la declaración** (a veces opcional): C — `int i=0, j=1;`; ADA — `I,J: INTEGER:=0;`.

Variables anónimas y referencias (punteros)

Algunos lenguajes permiten que el r-valor de una variable sea una **referencia** al l-valor de otra.

```
type pi = ^integer;
var pxi: pi;
new(pxi);          // aloca una variable anónima; pxi apunta a ella

type ppi = ^pi;    // puntero a un puntero
var ppxi: ppi;
new(ppxi);
```

ALIAS Y SOBRECARGA

Alias: distintos nombres → 1 entidad. Dos variables son alias si comparten el mismo objeto de dato en el mismo ambiente de referencia — si sus caminos de acceso conducen al mismo objeto, modificarlo por un camino lo modifica para todos los caminos.

```
int x = 5;
int *px = &x;
py = px;          // px y py son alias: apuntan al mismo x
```

El uso de alias puede llevar a difícil lectura y a errores: con `int *i=&x; int *j=&x;`, modificar `*i=10;` también cambia lo que ve `j` — esto es un **efecto lateral** (modificación de una variable no local).
Sobrecarga

Sobrecarga: 1 nombre → distintas entidades — lo opuesto al alias. Un nombre está sobrecargado si en un momento referencia más de una entidad y hay suficiente información para desambiguar la ligadura unívocamente.

```
int i,j,k; float a,b,c;
i = j + k;   // "+" resuelve a suma entera
a = b + c;   // "+" resuelve a suma de punto flotante
// los tipos permiten desambiguar el mismo nombre "+" en compilación
```

4

Unidades de Programa — Rutinas

Clase 4

Repaso de entidades y binding aplicado ahora a las **rutinas**: su propia tupla <Nombre, Alcance, Tipo, L-value, R-value>, cómo se comunican entre sí, cómo se representan en ejecución (procesador abstracto SIMPLESEM) y los tres modelos de manejo de memoria en ejecución.

REPASO RÁPIDO

Los lenguajes trabajan con **entidades** (variables, unidades, sentencias) que tienen **atributos**, y a las que hay que ligarles un valor antes de usarlas (**binding**, estático o dinámico) — ver Clase 3 para el detalle completo aplicado a la variable. Esta clase aplica la misma lógica a una nueva entidad: la **rutina**.

UNIDADES Y RUTINAS

Los lenguajes permiten que un programa esté compuesto por **unidades**: `UNIDAD → acción abstracta`. En general se las llama **rutinas**, que se dividen en:

✓ **Procedimientos:** no devuelven un valor.

✓ **Funciones:** devuelven un valor.
Hay lenguajes que SOLO tienen "funciones" y simulan los procedimientos con funciones que devuelven `void` (ej.: C, C++, Python).

LA TUPLA DE LA RUTINA: NOMBRE Y ALCANCE

LA TUPLA DE LA RUTINA: NOMBRE Y ALCANCE

Igual que la variable, la rutina se describe con <Nombre, Alcance, Tipo, L-value, R-value>.

- ✓ **Nombre:** string/identificador usado para invocar la rutina. Se introduce en su declaración; la llamada debe estar **dentro del alcance** del nombre.
- ✓ **Alcance:** rango de instrucciones donde se conoce el nombre — desde su declaración hasta algún constructor de cierre. Según el lenguaje, estático o dinámico.
- ✓ **Activación:** la llamada solo puede hacerse dentro del alcance de la rutina.

```
void uno() { dos(); }
void dos() { printf("hola"); }
uno();
// INCORRECTO en ANSI C: dos() no puede ser invocada por uno()
// porque dos NO está en su alcance (se declaró después)
```

DEFINICIÓN VS. DECLARACIÓN

Algunos lenguajes (C, C++, ADA) distinguen entre **declaración** (solo el encabezado — nombre, tipo de parámetros y de retorno) y **definición** (encabezado + cuerpo).

```
int sum(int n);          // declaración
int sum(int n) { ... }   // definición (encabezado + cuerpo)
```

Distinguirlas permite manejar **rutinas mutuamente recursivas**: se declara A, se define B (que ya puede llamar a A), y luego se define A (que puede llamar a B).
Cada lenguaje resuelve distinto qué pasa si no hay declaración previa: en C clásico se "asume" un prototipo que devuelve entero (puede dar error si el tipo real no coincide); en C++ da error directo ("no declarada"); en Pascal se usa la palabra `forward` para declarar antes de definir.

TIPO, L-VALUE Y R-VALUE DE LA RUTINA

- ✓ **Tipo (signatura):** tipo de los parámetros + tipo del valor de retorno. Una rutina con parámetros T1,T2...Tn y retorno R se anota `fun: T1xT2x...xTn → R`. Un llamado es correcto si hay **conformidad de tipos** entre parámetros formales y reales.
- ✓ **L-value:** el lugar de memoria donde se almacena el **cuerpo** de la rutina.
- ✓ **R-value:** la ejecución que dispara la llamada. Puede ser **estático** (el caso más usual, invocación directa por nombre) o **dinámico** (variables de tipo rutina, implementadas con punteros a función).

COMUNICACIÓN ENTRE RUTINAS

Dos vías: **ambiente no local** (variables visibles pero no locales a la rutina) y **parámetros** (dan datos distintos en cada llamado → mayor legibilidad y modificabilidad).

- ✓ **Parámetros formales:** los que aparecen en la definición de la rutina.
- ✓ **Parámetros reales:** los que aparecen en la invocación (pueden ser un dato o una rutina).

Ligadura entre parámetros formales y reales

- ✓ **Método posicional:** se ligan uno a uno según el orden — `Fi` se liga a `Ai` para `i` de 1 a n. Deben conocerse las posiciones. Variante: valores por defecto (C++: `int distancia(int a=0, int b=0)`).

✓ **Método por nombre:** se ligan por el nombre del formal, sin importar el orden (ADA: `Ejem(C=>Z, A=>X, B=>Y))` — deben conocerse los nombres de los formales; se puede combinar con posicional y con valores por defecto.

REPRESENTACIÓN EN EJECUCIÓN

La definición de la rutina especifica un proceso de cómputo; al invocarla se ejecuta una **instancia** — la representación de la rutina en ejecución — compuesta por:

✓ **Segmento de código:** las instrucciones de la unidad, en la memoria de instrucciones (C). Contenido fijo.

✓ **Registro de activación (RA):** los datos locales de la unidad, en la memoria de datos (D). Contenido cambiante.

Elementos en ejecución

✓ **Punto de retorno:** pieza cambiante de información que se salva en el RA de la unidad llamada, para saber a dónde volver.

✓ **Ambiente local:** variables locales, ligadas a los objetos de su propio RA.

✓ **Ambiente no local:** variables no locales, ligadas a objetos de los RA de otras unidades.

PROCESADOR ABSTRACTO SIMPLESEM

Modelo (semántica operacional) para entender el efecto de ejecutar cada instrucción del lenguaje, traduciéndolo a una secuencia equivalente en este procesador abstracto.

✓ **Memoria de Código (C):** `C(y)` = valor almacenado en la celda y-ésima (instrucciones).

✓ **Memoria de Datos (D):** `D(y)` = valor almacenado en la celda y-ésima; D(y) es el l-valor, C(y) o D(y) su r-valor.

✓ **ip:** puntero a la instrucción que se está ejecutando; arranca en 0 y se actualiza tras cada instrucción.

✓ Ciclo de ejecución: obtener la instrucción actual `C[ip]` → incrementar ip → ejecutarla.

Instrucciones

```
set target,source       // copia el valor de source en target
set 10,D[20]             // copia lo almacenado en la posición 20 en la posición 10
set 15,read              // E/S: lo leído se almacena en la dirección 15
set write,D[50]          // E/S: transfiere lo almacenado en 50 al exterior
set 99,D[15]+D[33]*D[4]  // combinación de expresiones

jump 47                  // bifurcación incondicional a la dirección 47 de C
jumpt 47,D[13]>D[8]      // bifurca solo si la expresión da verdadera

set D[10],D[20]
jump D[30]                // direccionamiento indirecto: salta a la dirección guardad
```

ESTRUCTURA DE EJECUCIÓN: 3 MODELOS DE MEMORIA

MODELO	ESPACIO	CARACTERÍSTICAS
Estático	Fijo	Se deduce todo del código antes de ejecutar; la alocación puede hacerse estáticamente; no puede haber recursión.

Basado en pila	Predecible	El espacio se deduce del código (Algol-60); sigue disciplina LIFO; variables se alocan/desalocan automáticamente según el alcance; la pila no es parte de la semántica del lenguaje, es parte del modelo.
Dinámico	Impredecible	Uso de memoria impredecible; los datos se alocan solo cuando se necesitan, en cualquier punto del programa; no puede modelarse con una pila; se alocan en la zona de memoria **heap**.

EJEMPLO PROGRESIVO: DE C1 A C2'

- ✓ **C1 — lenguaje simple:** sentencias y tipos simples (enteros, reales, arreglos, estructuras), sin funciones, datos estáticos de tamaño fijo. Un programa = una sola rutina `main()`. Todo su RA (zona de Datos) es **estático**: cada variable tiene una celda fija, conocida en compilación — y el código (zona Código) traduce cada sentencia a instrucciones SIMPLESEM (set/jump/jumpt).
- ✓ **C2 — C1 + rutinas internas:** programa = datos globales + declaraciones de rutinas + rutina principal (con sus propios datos locales, invocada automáticamente al ejecutar). Las rutinas internas son **disjuntas** (no anidadas) y **no recursivas**; su ambiente = datos locales + datos globales. Con todo estático, cada rutina tiene su propio bloque fijo de celdas en la zona de Datos.
- ✓ **C2 — modelo call-return:** al invocar una rutina, su segmento de código guarda en su RA el **punto de retorno** (la dirección a la que hay que volver) antes de saltar a ejecutarla, y salta a esa dirección al terminar.
- ✓ **C2' — compilación separada:** cada rutina puede compilarse en un archivo/módulo distinto, con direcciones **relativas** (el compilador no puede ligar variables locales ni globales a direcciones absolutas). El **Linkeditor** combina los módulos y liga la información faltante (direcciones reales). C2 y C2' no difieren semánticamente — solo en cómo se traducen y ensamblan.

6

Formas de comunicación entre las rutinas

Clase 6

Cómo comparten datos las rutinas entre sí: ambiente no local vs. parámetros, los distintos mecanismos de pasaje (por valor, por referencia, por nombre...), cómo los implementa cada lenguaje, y qué pasa cuando se pasa un subprograma como parámetro.

RUTINAS / SUBPROGRAMAS

Una rutina es una **Unidad de Programa** formada por un conjunto de sentencias que representan una **acción abstracta**: a nivel de diseño permite definir una operación creada por el usuario, a semejanza de las operaciones primarias del lenguaje — da modularidad, claridad y buen diseño. Responden al **esquema call/return**.

- ✓ **Procedimientos:** dan nombre a un conjunto de sentencias para resolver un subproblema; pueden no recibir ni devolver ningún valor; sus resultados los produce en variables no locales o en parámetros que cambian de valor.
- ✓ **Funciones:** además de ejecutar sentencias, devuelven un valor al punto donde se llamaron; se invocan dentro de expresiones y lo que producen reemplaza a la invocación; siempre deben retornar un valor. Semánticamente distintas de los procedimientos.

> **Conclusiones:** al diseñar un subprograma el programador se concentra en el *cómo*; al usarlo, en el *qué* (la implementación queda oculta). Con una sola definición se pueden crear muchas activaciones — la definición es un patrón para crear activaciones durante la ejecución. Codificar un

> subprograma es como incorporar una nueva sentencia al lenguaje.

CÓMO COMPARTEN DATOS LAS RUTINAS

Si las unidades usan solo variables locales no hay problema. Si necesitan datos que no son locales, hay dos formas:

- ✓ **Acceso al ambiente no local** — puede llevar a más errores (variables visibles donde no deberían):
  - ✓ *Implícito:* automático, según la regla de alcance del lenguaje (estático o dinámico).
  - ✓ *Explícito (ambiente común):* se definen áreas comunes de código y el programador especifica qué comparte — cada lenguaje tiene su forma: paquetes/TDA de ADA, variables externas de PL/1, cláusula `COMMON` de Pascal.
- ✓ **Uso de parámetros** — la forma mejor y más clara: da distintos datos en cada llamado, mayor legibilidad y modificabilidad.

Por qué el pasaje de parámetros es mejor

- ✓ **Protección:** el uso intensivo de accesos al ambiente no local decrementa la seguridad y legibilidad — las variables terminan siendo visibles donde no deberían.
- ✓ **Legibilidad:** permite encontrar más fácilmente los errores — al transformar en rutinas o funciones a las que se les pasan valores, es más fácil depurar sin chequear cada repetición del código.
- ✓ **Modificabilidad:** ante un error, uno se enfoca en qué se está compartiendo (qué argumentos/parámetros y de qué tipo).

Una buena cualidad de un programador es **minimizar el acceso a datos no locales**.

PARÁMETROS: FORMALES, REALES, EVALUACIÓN Y LIGADURA

- ✓ **Parámetro real (argumento):** valor u otra entidad usada para pasar a un procedimiento o función — está en la parte de la **invocación**.
- ✓ **Parámetro formal:** variable usada para recibir valores de entrada en la rutina — está en la parte de la **declaración**. Es una variable local a su entorno.

Un parámetro real puede ser un valor, entidad, expresión u otro, y puede ser local, no local o global a quien hace la llamada — depende de cada lenguaje.

Evaluación y ligadura

- ✓ **Evaluación:** en general, antes de la invocación se evalúan los parámetros reales y luego se hace la ligadura — se verifica que todo esté bien antes de transferir el control.
- ✓ **Ligadura posicional:** se corresponden con la posición que ocupan en la lista — deben ir en el mismo orden.
- ✓ **Ligadura por nombre/palabra clave:** se corresponden con el nombre del formal, por lo que pueden colocarse en distinto orden — desventaja: hay que conocer los nombres de los formales (riesgo de error). En ADA pueden mezclarse ambos métodos; en C++/ADA/Python/JavaScript los formales pueden tener valores por defecto, sin necesidad de listarlos todos en la invocación.

```
// Python — ligadura por nombre
P(y => 4, x => z);   // Procedure P(x: IN integer, y: IN float) — orden distinto al d
```

TIPOS DE PARÁMETROS

Lo que se pasa como parámetro puede ser de dos clases, cada una con sus propios subtipos:

✓ **Datos** — según el sentido en que fluye la información: modos IN, OUT, IN/OUT (desarrollados abajo).

✓ **Subprogramas** — se pasa el nombre de una rutina como parámetro (desarrollado más abajo).

MODOS DE PASAJE DE DATOS

Según el punto en que se conecta el parámetro formal con el real:

MECANISMO	CONEXIÓN	CÓMO FUNCIONA	VENTAJA	DESVENTAJA
Por valor (IN)	Al inicio	El valor del real inicializa al formal; el formal actúa como variable local nueva; se corta la vinculación. Mecanismo por defecto en la mayoría de los lenguajes.	Protege el real, sin efectos colaterales	Consume tiempo y espacio al copiar
Por valor constante (IN)	Al inicio	Igual que por valor, pero la implementación verifica que el formal no sea modificado dentro de la rutina (ej.: `const` en C/C++, modo IN de ADA).	Protege también contra modificación accidental	Requiere más trabajo de implementación
Por resultado (OUT)	Al final	El formal es una variable local sin valor inicial (hay que inicializarla); al terminar la rutina, su valor final se copia al real.	Protege el real durante toda la ejecución	Consume tiempo/espacio; riesgo si no se inicializa
Por valor-resultado (IN/OUT)	Al inicio *y* al final	Copia al entrar (como por valor) y copia en sentido inverso al salir (como por resultado); durante la ejecución el formal y el real NO están ligados entre sí.	Combina las ventajas de valor y resultado	Con alias puede dar resultados distintos a por referencia (ver ejemplo)
Por referencia (IN/OUT)	Durante toda la ejecución	El formal es la **dirección (l-valor)** del real: es una referencia local a un ambiente no local — se trabaja directamente sobre el mismo dato (alias). También llamado "por variable".	Eficiente: no copia datos, ideal para grandes volúmenes	Acceso más lento (indirección); genera alias que afectan legibilidad y dificultan la depuración
Por nombre (IN/OUT)	Se reevalúa en cada uso	El formal se sustituye textualmente por una expresión que apunta al real, más un	Máxima flexibilidad (evolución de valor diferida)	Lento (se reevalúa cada vez), difícil de implementar, poco usado hoy

		puntero a su entorno; la ligadura del valor se difiere hasta que se usa (evaluación diferida). Se implementa con **thunks** (procedimientos sin nombre).		

EL CASO CLÁSICO: ALIAS ENTRE POR VALOR-RESULTADO Y POR REFERENCIA

Mismo código, resultado distinto según el mecanismo — clave para entender la diferencia:

```
Procedure A();
  var m: integer;
  Procedure B(modo j: integer);
    begin
      j := j+5;   j := j+m;
      write(j, m);
    end;
  begin
    m := 4;   B(m);
    write(m);
  end;
```

✓ **Por valor-resultado:** j copia m (4) → j=9 → j:=j+m (m sigue en 4, sin cambios) → j=13. Adentro imprime **"13 4"**. Al terminar B, se copia j (13) al real m → afuera imprime **13**.

✓ **Por referencia:** j y m son el mismo lugar de memoria (alias). j:=j+5 → m también pasa a 9. j:=j+m usa el m ya actualizado (9) → j=18, m=18 al mismo tiempo. Adentro imprime **"18 18"**. Afuera imprime **18**.

La diferencia se debe a que por valor-resultado copia y desliga durante la ejecución (m no ve los cambios de j hasta el final), mientras que por referencia mantiene el alias todo el tiempo: cualquier cambio en j se refleja en m de inmediato.

PASAJE DE PARÁMETROS SEGÚN EL LENGUAJE

LENGUAJE	MECANISMO(S)
C	Por valor (único mecanismo nativo); "por referencia" se simula con punteros; por valor constante agregando `const`.
Pascal	Por valor (por defecto); por referencia opcional con `var`.
C++	Similar a C, pero con más uso de por referencia (&).
Java	Solo por valor — pero como las variables de tipos no primitivos son referencias a objetos en el heap, copiar esa referencia produce en la práctica un efecto de "por referencia" sobre el objeto.
PHP	Por valor (predeterminado); por referencia con &.
Ruby	Por valor; pero si el objeto pasado es "mutable" no se copia, se trabaja directamente sobre él.
Python	Envía objetos: si son inmutables se comporta como por valor; si son mutables (ej. listas) no se copia, se trabaja sobre el objeto.

ADA	Modo IN por copia (defecto), modo OUT por resultado, IN-OUT: para tipos primitivos es por valor-resultado, para tipos no primitivos/compuestos (arreglos, registros) es por referencia. En funciones solo se permite por copia de valor, para evitar efectos colaterales.

SUBPROGRAMAS COMO PARÁMETRO

Algunos lenguajes permiten pasar el **nombre de un subprograma** como parámetro (ej.: pasarle a una rutina de ordenamiento cuál es el criterio de comparación a usar). No lo incorporan todos los lenguajes — ADA no contempla subprogramas como valores (usa unidades genéricas en su lugar); Pascal permite pasar una referencia a un procedimiento; C permite pasaje de funciones como parámetros.

Ambiente de referencia para las variables no locales del subprograma pasado

Al ejecutarse el subprograma recibido como parámetro, ¿cuál es su ambiente de referencia para las variables no locales? Hay tres opciones:

- ✓ **Superficial (shallow):** el ambiente es el de la unidad que tiene (recibe) el parámetro formal subprograma. Ej.: SNOBOL.
- ✓ **Profundo (deep):** el ambiente es el de la unidad donde está **declarado** el subprograma usado como parámetro real. Se usa en lenguajes con alcance estático y estructura de bloque — es el más natural, porque el entorno queda determinado por la posición léxica de su definición.
- ✓ **Ad hoc:** el ambiente es el de la unidad donde se encuentra el **llamado** a la unidad que recibió el parámetro subprograma. Poco natural; nunca se usa en la práctica.

El enlace superficial no es apropiado para lenguajes de alcance estático con subprogramas anidados: el receptor puede no estar en el entorno estático de quien envía el parámetro, así que no sería natural que tenga acceso a sus variables. Cuando el mismo subprograma que declara una rutina también la pasa como parámetro, superficial y profundo coinciden.

Ejemplo (JavaScript)

`sub1` declara `sub2` (usa `x`), `sub3` y `sub4(subx)`. Se ejecuta `sub3()`, que llama `sub4(sub2)`, que ejecuta `subx()` (es decir, `sub2`). Cada regla da un resultado distinto para el `x` que ve `sub2`:

- ✓ **Superficial:** el entorno es el de `sub4` (quien tiene el parámetro formal) → imprime **4**.
- ✓ **Profundo:** el entorno es el de `sub1` (donde está declarado `sub2`) → imprime **1**.
- ✓ **Ad hoc:** el entorno es el de `sub3` (quien hizo el llamado) → imprime **3**.

UNIDADES GENÉRICAS

Una unidad genérica puede instanciarse con parámetros formales de distinto tipo — ej.: una unidad que ordena elementos de un arreglo, instanciable para enteros, flotantes, etc. Como pueden usarse diferentes instancias con diferentes subprogramas, proveen la misma funcionalidad que un parámetro subprograma sin pasar realmente una rutina como valor.

```
// ADA
generic
  type Elemento is private;
package Conjuntos is
  // ... operaciones genéricas sobre Conjunto ...
end Conjuntos;

package Conjuntos_Reales  is new Conjuntos (Float);
package Conjuntos_Enteros is new Conjuntos (Integer);
```

7

Tipos de Datos

Clase 7

Qué es un tipo de dato, sus dos grandes familias (predefinidos vs. definidos por el usuario) y los constructores para armar tipos compuestos, punteros y manejo de memoria, el concepto de Tipo Abstracto de Dato (TAD) como evolución hacia la Clase, y el Sistema de Tipos de un lenguaje.

CONCEPTO E HISTORIA

Un **tipo** = un conjunto de valores + un conjunto de operaciones para manipularlos. No es solo una colección de datos: organiza los datos y tiene comportamiento semántico.

Evolución histórica: los primeros lenguajes solo tenían unos pocos tipos básicos → aparecen los **tipos definidos por el usuario** (buscando legibilidad y modificabilidad) → evolucionan hacia el **Tipo Abstracto de Dato (TAD)**, que separa representación de operaciones → la evolución del TAD es el concepto de **Clase**.

PREDEFINIDOS VS. DEFINIDOS POR EL USUARIO

	ELEMENTALES	COMPUESTOS
Predefinidos	enteros, reales, caracteres, booleanos	string
Definidos por el usuario	enumerados	arreglos, registros, listas, etc.

Dominio de un tipo = conjunto de valores posibles de ese tipo.

TIPOS PREDEFINIDOS

Reflejan el comportamiento del hardware subyacente. Ventajas: invisibilidad de la representación, verificación estática, desambiguar operadores, control de precisión.

Aspectos a definir para cada uno: números enteros/reales, caracteres, booleano, valores y longitud máxima, conversiones. Algunos aspectos quedan **definidos por la implementación** y otros **definidos por el lenguaje**.

TIPOS DEFINIDOS POR EL USUARIO

Permiten especificar agrupaciones de datos elementales y, recursivamente, agregaciones de agregados, mediante **constructores**. Separan especificación de implementación.

- ✓ **Legibilidad:** nombres apropiados, ej. `dias[0..31]`.
- ✓ **Estructura jerárquica de refinamiento:** un tipo se arma a partir de otros ya definidos.
- ✓ **Modificabilidad** y **factorización** del diseño.
- ✓ **Instanciación:** el tipo es una descripción abstracta; los detalles de implementación quedan ocultos en su definición.

```
// pseudocódigo
Record Persona { String nombre, apellido; int edad; ... }
```

```
Persona vecino = new Persona(...);
vecinos = array[1..10] of Persona;
```

CONSTRUCTORES DE TIPOS COMPUESTOS

Cuatro mecanismos: Producto Cartesiano, Correspondencia Finita, Unión (y Unión Discriminada), Recursión.

- ✓ **Producto Cartesiano** (A1×A2×...×An): conjunto de n-tuplas ordenadas → **registro/struct**. Ej. en C: `typedef struct { int nro_lados; float tamaño_lado; } reg_poligon;` — definición, campos, instancia con valor compuesto inicial, acceso por notación puntual.
- ✓ **Correspondencia Finita:** función de un conjunto finito de valores de un tipo dominio (DT) en valores de un tipo rango (RT); si DT es subrango de enteros → **arreglo** accesible por subíndice. Ej. C: `char digits[10]` (dominio [0..9], resultado char, acceso `digits[i]`); Python: `l = ["una lista", [1, 2]]; mi_var = l[1][0] # vale 1`.
- ✓ **Unión / Unión Discriminada:** define un tipo como la disjunción de los tipos dados; permite manipular diferentes tipos en distinto momento de la ejecución, con chequeo dinámico. La unión discriminada agrega un **discriminante** que indica la opción elegida y permite manejarla en forma segura. Problemas: discriminante y variantes pueden manejarse independientemente, la implementación puede ignorar los chequeos, y puede omitirse el discriminante (con lo cual no se puede chequear aunque se quisiera).
- ✓ **Recursión:** un tipo recursivo T se define como una estructura que puede contener componentes del tipo T. Define datos agrupados cuyo tamaño puede crecer arbitrariamente y cuya estructura puede ser arbitrariamente compleja. Los lenguajes convencionales la implementan a través de **punteros**.

```
// C — union y unión discriminada
union address{ //campos mutuamente exclusivos
  short int offset;
  long unsigned int absoluto;
};
enum descriptor {abs, off}
typedef struc {
  address location;
  descriptor kind;
} address_seguro;
```

PUNTEROS

Un **puntero** es una referencia a un objeto; una **variable puntero** es una variable cuyo r-valor es una referencia a un objeto. Ventajas: estructuras de tamaño arbitrario (número de ítems no determinado), relaciones múltiples entre ítems sin duplicar datos, acceso a bajo nivel. Los punteros amplían el rango de celdas de memoria referenciables y el tipo de valores que puede contener un objeto — se los compara con los `go to` (que amplían el rango de sentencias ejecutables) por ampliar de forma similar el acceso.
Por acceder a bajo nivel, pueden obscurecer o hacer inseguros a los programas que los usan.
Inseguridad de los punteros

- ✓ **Violación de tipos:** aritmética de punteros que hace que p termine interpretando bytes de un tipo como si fueran de otro (ej. C: incrementar un `int*` y usarlo para leer/escribir un `float`).
- ✓ **Referencias sueltas (dangling):** puntero que contiene la dirección de una variable dinámica ya desalocada; si luego se usa, produce error.
- ✓ **Punteros no inicializados:** peligro de acceso descontrolado a memoria. Se mitiga con verificación

dinámica de inicialización o con un valor especial nulo (`nil` en Pascal, `void` en C/C++, `null` en ADA/Python).

- ✓ **Punteros y uniones discriminadas:** mismo efecto que la aritmética de punteros. Java elimina la noción de puntero explícito para evitarlo.
- ✓ **Alias:** dos o más punteros pueden apuntar al mismo objeto (ej. `p1 = &x; p2 = &x;` → p1, p2 y x son alias entre sí).
- ✓ **Liberación de memoria — objetos perdidos:** las variables puntero se alocan como cualquier otra en la pila de RA; los objetos apuntados alocados con `new` van al **heap**, que puede agotarse si no se libera lo alocado.

MANEJO DE MEMORIA — LIBERACIÓN

Un objeto es **accesible** si alguna variable en la pila lo apunta directa o indirectamente; es **basura (garbage)** si no es accesible. Si los objetos del heap dejan de ser accesibles, esa memoria podría liberarse.

- ✓ **Liberación explícita** (responsabilidad del programador): `dispose` en Pascal, `delete` en C++. Puede generar referencias sueltas (ej.: `new(p); q:=p; dispose(p); ...q...` → q queda dangling).
- ✓ **Liberación implícita** (independiente de la aplicación): el sistema descubre la basura en ejecución con un algoritmo de **garbage collector**. Clave en lenguajes de uso intensivo de variables dinámicas (LISP, Python). Debe ser eficiente y, si el sistema es interactivo o de tiempo real, ejecutar en paralelo sin degradar el rendimiento.
- ✓ Algol-68 y Simula 67 lo reclaman automáticamente; Eiffel y Java (todos los objetos referenciados por punteros) proveen recolector automático; ADA chequea dinámicamente que el tiempo de vida del objeto apuntado sea ≤ el del puntero; PHP chequea una cuenta de referencias.

	JAVA	PHP
Mecanismo	Generacional	Cuenta de Referencias
Reconocimiento	Divide la Heap en 3 generaciones (Joven / Vieja / Permanente)	Contenedor z-val con 2 campos: is_ref y refcount
Algoritmo	Envejece y compacta los objetos referenciados; elimina los sin referencia y reorganiza los bloques de generaciones	Cuenta las referencias de cada objeto; al llegar a 0 y no ser referenciado, se elimina
Configuraciones	Serial GC, Parallel GC, CMS GC, Garbage First	Fallo con fuga de memoria solucionado a partir de 5.3.0

TIPO ABSTRACTO DE DATO (TAD)

Abstracción = mecanismo para manejar la complejidad: representar algo descubriendo sus características esenciales y suprimiendo las que no lo son. Principio básico: la **información oculta**.

TAD = Representación (datos) + Operaciones (funciones y procedimientos). Los tipos de datos son abstracciones; construir nuevos tipos es **abstracción de datos**; los nuevos tipos definidos por el usuario se llaman **tipos abstractos de datos**.

Un TAD debe satisfacer: **encapsulamiento** (representación + operaciones descritas en una única unidad sintáctica, refleja las abstracciones descubiertas en el diseño) y **ocultamiento de la información** (representación e implementación permanecen ocultas, refleja los niveles de abstracción y da modificabilidad).

Unidades de programación que pueden implementar un TAD según el lenguaje: Simula-67 `class` (estructura sintáctica única, pero no satisface el ocultamiento), Modula-2 módulo, Ada paquete, C++ clase, Java clase.

Especificación de un TAD

Conjunto de axiomas que describen el comportamiento de todas las operaciones. Incluye **sintaxis** (`Operación(Tipo argumento, ...) → Tipo resultado`) y **semántica** (`Operación(valores particulares) ⇒ expresión resultado`). Las operaciones definidas por sí mismas que se eligen como constructor suelen ser las que inicializan.

```
// Ejemplo TAD Conjunto (sintaxis + semántica)
Conjuntovacio → Conjunto
Añadir(Conjunto, Elemento) → Conjunto
Retirar(Conjunto, Elemento) → Conjunto
Pertenece(Conjunto, Elemento) → boolean
Esvacio(Conjunto) → boolean
Cardinal(Conjunto) → entero
Union(Conjunto, Conjunto) → Conjunto
// semántica, para todo e1,e2 y C,D Conjunto:
Añadir(Añadir(C,e1),e1) ⇒ Añadir(C,e1)
Retirar(Conjuntovacio,e1) ⇒ Conjuntovacio
```

```
// Ejemplo TAD Pila en ADA – especificación, encapsula y oculta
package PILA is
  type PILA limited private
  MAX: constant := 100
  function EMPTY (P:in PILA) return boolean
  procedure PUSH (P:inout PILA, ELE:in INTEGER)
  procedure POP (P: inout PILA)
  procedure TOP (P:in PILA) return INTEGER
private
  type PILA is
    vecpila : array (1..MAX) of INTEGER
    tope: INTEGER range 0..MAX := 0
end PILA;
// instanciación: aloca y ejecuta el código de inicialización
pil: PILA;
pil.PUSH(pil, y);
```

TAD: Clases. En términos prácticos, una clase es un tipo definido por el usuario que contiene la especificación de los datos que describen un objeto junto con la descripción de las acciones que un objeto conoce (**Atributos + Métodos**). Agrega un segundo nivel de abstracción: agrupar clases en jerarquías, donde la clase hereda las propiedades de la superclase.

SISTEMA DE TIPOS

Conjunto de reglas usadas por un lenguaje para estructurar y organizar sus tipos. Su objetivo es escribir programas seguros; conocerlo permite entender mejor los aspectos semánticos del lenguaje.

- ✓ Provee mecanismos de expresión: expresar tipos intrínsecos o definir tipos nuevos, y asociar los tipos definidos con construcciones del lenguaje.
- ✓ Define reglas de resolución: **equivalencia** de tipos (¿dos valores tienen el mismo tipo?), **compatibilidad** de tipos (¿puedo usar el tipo en este contexto?), **inferencia** de tipos (¿qué tipo se deduce del contexto?). Cuanto más flexible el lenguaje, más complejo el sistema: **seguridad vs. flexibilidad**.

Tipado fuerte vs. tipado débil

El sistema es **fuerte** cuando especifica restricciones sobre cómo pueden operarse valores de tipos distintos; si no, es **débil**.

```
// tipado débil: convierte implícitamente
a = 2; b = "2"
Concatenar(a,b) // retorna "22"
Sumar(a,b)      // retorna 4

// tipado fuerte: exige conversión explícita
Concatenar(a,b)         // error de tipos
Concatenar(str(a),b)    // retorna "22"
Sumar(a,int(b))         // retorna 4
```

Un lenguaje es **fuertemente tipado (type safety)** si el sistema de tipos impone restricciones que aseguran que no se producirán errores de tipo en ejecución, y si todos los errores de tipo se detectan. La intención es evitar errores de aplicación tolerando errores del lenguaje, detectados lo antes posible.
Python: fuertemente tipado y tipado dinámico. C: débilmente tipado y tipado estático. Gobstone: fuertemente tipado y tipado estático.
Tipo y tiempo de chequeo (ligadura)

- ✓ **Tipado estático:** ligaduras en compilación — exige tipos predefinidos, todas las variables declaradas con tipo asociado, y todas las operaciones con tipos de operandos y resultado especificados.
- ✓ **Tipado dinámico:** ligaduras en ejecución, más comprobaciones en tiempo de ejecución.
- ✓ **Tipado seguro:** ni estático ni inseguro.

Los errores pueden ser del **lenguaje** (sintácticos o semánticos, en el uso del lenguaje) o de la **aplicación** (en el comportamiento); los lenguajes permiten detectarlos y/o prevenirlos.
Reglas de equivalencia y conversión

- ✓ **Equivalencia por nombre:** dos variables son del mismo tipo si están declaradas juntas o con el mismo nombre de tipo.
- ✓ **Equivalencia por estructura:** dos variables son del mismo tipo si los componentes de su tipo son iguales.
- ✓ Un tipo es **compatible** con otro si es equivalente o si se puede convertir (**coerción**): reglas del lenguaje según el tipo de los operandos y su jerarquía.
- ✓ **Widening** (ensanchar): cada valor del dominio tiene su correspondiente en el rango, sin pérdida (ej. entero → real; Pascal solo admite este sentido).
- ✓ **Narrowing** (estrechar): puede haber pérdida de información (ej. real → entero); algunos lenguajes avisan la pérdida; en C depende del contexto con coerción simple.
- ✓ **Casting:** conversión explícita, se fuerza a que se convierta.

Reglas de inferencia y nivel de polimorfismo

La inferencia permite que el tipo de una entidad declarada se deduzca en lugar de ser declarado explícitamente, a partir de: un operador predefinido, un operando, un argumento, o el tipo del resultado.

- ✓ **Mono-mórfico:** cada entidad se liga a un único tipo (estático) — una función debe implementarse para cada tipo particular.
- ✓ **Polimórfico:** las entidades pueden ligarse a más de un tipo. Variables polimórficas toman valores de distintos tipos; operaciones polimórficas aceptan operandos de varios tipos; los tipos polimórficos tienen operaciones polimórficas.
- ✓ **Polimorfismo universal:** una única operación se aplica uniformemente sobre un conjunto de tipos relacionados. Incluye el **paramétrico** (uniformidad dada por parámetros — tipo parametrizado, ej. `lista(T) = T*`) y el **por inclusión** (modela subtipos y herencia: un subtipo T' de T puede definirse como

subconjunto de los valores de T con el mismo conjunto de operaciones; la herencia permite definir una clase derivada agregando atributos y comportamiento).

✓ **Polimorfismo ad-hoc:** una función se aplica a distintos tipos con comportamiento sustancialmente diferente en cada caso. Incluye **sobrecarga** (abstracciones distintas ligadas al mismo símbolo/identificador) y **coerción** (un operador que espera un tipo T puede aplicarse de forma segura sobre un operando de tipo diferente).

8

Estructuras de Control

Clase 8

El medio por el cual los programadores determinan el flujo de ejecución entre los componentes de un programa: a nivel de **sentencia** (secuencia, selección, iteración) y a nivel de **unidad** (pasaje de parámetros, call-return, excepciones — se ven en otras clases).

SECUENCIA

El flujo de control más simple: ejecución de una sentencia a continuación de otra. El delimitador más general y usado es `;`. Hay lenguajes **orientados a línea** que no tienen delimitador y establecen 1 instrucción por línea (Fortran, Basic, Ruby, Python).

Otros permiten **sentencias compuestas**: agrupar varias sentencias en una con delimitadores — Begin/End (ADA, Pascal) o `{ }` (C, C++, Java).

SENTENCIA DE ASIGNACIÓN

Asignación: sentencia que produce cambios en los datos de la memoria — asigna al **l-valor** de un objeto de dato el **r-valor** de una expresión (`x = a + b`).

SINTAXIS	LENGUAJES
A := B	Pascal, ADA, etc.
A = B	Fortran, C, Prolog, Python, Ruby, etc.
MOVE B TO A	COBOL
A ← B	APL
(SETQ A B)	LISP

DISTINCIÓN ENTRE SENTENCIA DE ASIGNACIÓN Y EXPRESIÓN

En cualquier lenguaje convencional existe diferencia entre sentencia de asignación y expresión: `(x + 3)*2` y `a > b && c < d` son expresiones; `y = (x + 3)*2;` es una sentencia.

Las sentencias de asignación **devuelven el valor** de la expresión y modifican la posición de memoria. En C la sentencia de asignación se define como una **expresión con efectos colaterales**: C evalúa de derecha a izquierda (`a=b=c=0;` asigna 0 a los tres), y una asignación dentro de una condición es válida y devuelve su valor (`if (i=30) printf("Es verdadero")` — no es comparación, == compara; esto es un error frecuente al confundir asignación con comparación). La mayoría de los lenguajes exige que a la izquierda de la asignación haya un l-valor; C no permite cualquier expresión ahí (ej. `++p = *q;` o `(i<j?z:y)=4;` fallan por requerir l-valor).

El **operador condicional** `?:` se escribe `a ? b : c` (a expresión booleana; b y c expresiones o sentencias): si a es verdadero devuelve b, si es falso devuelve c.

SELECCIÓN — IF

Estructura que permite expresar una elección entre un cierto número de sentencias alternativas (ejecución condicional).

- ✓ **If lógico de Fortran:** `If (condición lógica) sentencia` — si la condición es verdadera, ejecuta la sentencia.
- ✓ **if-then-else de ALGOL:** `if (cond) then sentencia1 else sentencia2`, permite 2 caminos. Anidados con múltiples caminos, generó el **problema de ambigüedad del "dangling else"**: el lenguaje no establecía cómo se asociaban los else con los if abiertos (ej. `if x>0 then if x<10 then x:=0 else x:=1000` — ¿a qué if pertenece el else?).
- ✓ **Solución 1 — regla de asociación:** cada else se empareja con el if solitario más próximo (buscando hacia atrás), cerrando siempre el último if abierto. Elimina la ambigüedad, pero las declaraciones anidadas pueden ser difíciles de leer sin buena sangría (Pl/1, Pascal, C).
- ✓ **Solución 2 — sentencia de cierre del bloque if:** `fi` (Algol 68), `end if` (ADA), end (Modula-2).
- ✓ **Solución 3 — instrucción compuesta:** bloque de instrucciones acotado entre `begin/end`, según el lenguaje.

If en C

No usa `then`; usa `( )` obligatorias en la condición y `{ }` (no begin/end) para bloques. Se recomienda usar siempre llaves, aunque sea una sola sentencia, para mayor legibilidad y mantenibilidad. Pascal, en cambio, no usa paréntesis en la condición.

SELECCIÓN — SENTENCIA CONDICIONAL CORTA

Operador ternario en C: `cond ? b : c;` — si cond es verdadero devuelve b, si es falso devuelve c (permite reemplazar un if largo por una línea).

En Python: `A if C else B` (equivalente a `C ? A : B` de C) — devuelve A si se cumple C, si no devuelve B. Python evalúa las condiciones con **evaluación con circuito corto**.
Circuito corto: la conjunción (and) da verdadero solo si ambos términos lo son — si el primero es falso, no evalúa el segundo (el resultado ya es falso). La disyunción (or) da falso solo si ambos lo son — si el primero es verdadero, no evalúa el segundo. Sirve para evitar errores al evaluar expresiones indefinidas (ej. `A and B` con B nulo) o cuando evaluar es costoso. Ej.: `a and false and f()` nunca llama a `f()`.

SELECCIÓN MÚLTIPLE

Los lenguajes incorporan sentencias de selección múltiple para elegir entre dos o más opciones, reemplazando cadenas del tipo `if (A) then s1 else if (B) then s2 else if ... else sn;`.

LENGUAJE	SINTAXIS Y REGLAS
Pascal	`case selector of valor1: s1; valor2: s2; else s4; end;` — el selector es de tipo ordinal (tiene predecesor/sucesor salvo extremos); no importa el orden de los casos; `else` opcional pero sin él es inseguro (no define qué pasa si el valor no matchea ninguna rama); se cierra con `end;` (no corresponde a ningún begin).

ADA	`case expr is when v1 => s1; when others => s2; end case;` — expresión solo entera o enumerativa; deben cubrirse "todos" los valores posibles (si no, no compila) salvo que se use `others` (debe ser la única y la última alternativa); se puede especificar rango con `..` y varias opciones con `|`.
C, C++	`switch (expr) { case v1: s1; break; default: s2; }` — cada rama etiquetada por valores constantes (enteros o char); si coincide, ejecuta y sigue con las siguientes ramas **salvo** exista `break` (que provoca la salida); `default` es opcional.
Ruby	`case expr when v1 ... else ... end` — la expresión puede ser cualquier valor (cadena, numérico, expresión); busca hasta que when coincida; si no coincide con ninguna va al `else` (opcional — si se omite y no matchea nada, sigue la ejecución sin asignar valor a la variable, buena práctica: siempre poner else con acción o comentario justificando por qué no).
PL/I	`SELECT (id); WHEN (v1) s1; OTHERWISE s2; END;` (Formato 1, con identificador) o con condiciones en cada `WHEN` sin identificador (Formato 2).

ITERACIÓN

Permite que una serie de acciones (el **cuerpo del bucle**) se ejecuten repetidamente (loop). Se agrupan comúnmente en dos tipos: **for** (se conoce el número de repeticiones al inicio del bucle) y **while** (el cuerpo se ejecuta mientras se cumpla una condición; hay 2 variantes según cuándo se chequea).

ITERACIÓN TIPO FOR

- ✓ **Fortran (Do):** `Do label var-de-control = valorIni, valorFin ... label continue`. La variable de control solo toma valores enteros y se incrementa en 1 salvo se indique otra cosa; `continue` con su etiqueta es la última sentencia del DO. El Fortran original evaluaba el límite al final del bucle (por eso "siempre se ejecutaba una vez" y traía problemas); desde Fortran 77 se evalúa antes de entrar. La variable de control nunca debe modificarse dentro del ciclo (genera errores de lógica).
- ✓ **Pascal, ADA, C, C++ (For):** `for loop_ctr_var := lower_bound to upper_bound do statement`. La variable de control puede tomar cualquier valor ordinal, no solo enteros. Pascal estándar no permite modificar los valores de límite inferior/superior ni de la variable de control dentro del bucle (da error de compilación); el valor de la variable fuera del bloque se asume indefinido — pero si se la modifica desde fuera del ciclo (no dentro), Pascal sí lo permite y genera efecto colateral en la ejecución.
- ✓ **ADA (loop/end loop):** encierra todo proceso iterativo entre `loop` y `end loop`; permite `Exit` para salir. La variable de control (iterador) no necesita declararse — se declara implícitamente al entrar al bucle y desaparece al salir. `in` indica incremento; permite decrementar con `in reverse`: `for i in 1..N loop V(i):=0; end loop`.
- ✓ **C, C++ (For):** 3 partes — inicialización (estado inicial), 1ra expresión (test, se evalúa antes de cada iteración; si no se cumple sale del rango), 2da expresión (incremento, se realiza después de cada iteración): `for(int x = 0; x < 100; x++){ println(x); }`. En C++ se puede declarar la variable dentro del for, con alcance hasta el final del bloque que encierra el for. Si se omiten una o ambas expresiones se puede crear un bucle sin fin, del que solo se sale con `break`, `goto` o `return`: `for ( ; ; ) { ... }`.
- ✓ **Python (For):** dos formas. Para **iterar sobre una secuencia** (lista, tupla, conjunto, diccionario, etc.): `for elem in iterable: código` — elem toma el valor de cada elemento; finaliza cuando se recorren todos. Para **iterar sobre un rango de valores**: función `range(max)` (de 0 a max-1), `range(min,max)`, o `range(min,max,step)` (incrementa de step en step, simula el for con variable de control).

ITERACIÓN TIPO WHILE

Chequeo al inicio (while): repite un proceso mientras se cumpla una condición; la condición se evalúa antes de entrar al proceso (puede no ejecutarse nunca).

LENGUAJE	SINTAXIS
Pascal	`while condición do sentencia begin ... end`
C, C++	`while (condición) sentencia; {....}`
ADA	`while condición sentencia ... end loop;`
Python	`while condición: sentencia1 ... sentencia n` (break para salir cuando se desea)

Chequeo al final (until-repeat / do-while): repite un proceso "hasta" que se cumpla una condición — ejecuta mientras NO se cumpla (sea falsa). La condición se evalúa al final, por lo que el proceso se ejecuta **al menos 1 vez**.

LENGUAJE	SINTAXIS
Pascal	`repeat sentencia ... until condición;`
C, C++	`do sentencia; ... while (condición);`

ITERACIÓN: LOOP DE ADA

ADA tiene además una estructura iterativa genérica, combinable con FOR y WHILE: `[identificador_bucle:] loop secuencia_de_sentencias end loop [identificador_bucle];` — el bucle dura indefinidamente si no se acota.

Se sale normalmente con `exit when condición;` dentro del loop, o encadenando una alternativa con cláusula `exit`. También puede abandonarse si algún procedimiento levanta una excepción.

10

Excepciones

Clase 10

Qué es una excepción y qué debe proveer un lenguaje para manejarlas, los dos modelos posibles de retorno (Reasunción vs. Terminación), y cómo los implementan PL/I, ADA, C++, CLU, Java, Python y PHP. (No hay Clase 9 en el material del curso.)

CONCEPTO

Los programadores suelen escribir bajo la suposición optimista de que nada saldrá mal, pero situaciones como división por cero, desbordamiento de pila, acceso a zona de memoria no permitida o índices fuera de rango ocurren con frecuencia. Una **excepción** es una condición inesperada o inusual que ocurre durante la ejecución del programa y no puede ser manejada en el contexto local: un comportamiento anómalo e indeseable que supuestamente ocurre raramente pero es necesario controlar.

Caso Ariane 5 (4/6/1996): el cohete explotó a los 37 segundos por una excepción de software no manejada — el software heredado del Ariane 4 generó valores más grandes de los que podía manejar, provocando un desbordamiento entero (integer overflow) al convertir un número de punto flotante a entero de 16 bits; la excepción no estaba controlada y se ejecutaron otras acciones. Más de 10 años

de fabricación y más de 7 millones de U$S perdidos.

Estas cuestiones pueden ser atendidas por mecanismos del **lenguaje** (construcciones especializadas, excepciones), del **hardware** (interrupciones) o del **Sistema Operativo** (comunicación entre procesos/IPC, señales). La excepción interrumpe el flujo normal de ejecución y ejecuta un controlador registrado previamente; no todos los lenguajes lo implementan igual.

QUÉ DEBE PROVEER UN LENGUAJE

Mínimamente: un modo de **definir** las excepciones, una forma de **reconocerlas**, una forma de **lanzarlas y capturarlas**, una forma de **manejarlas** especificando código y respuestas, y un **criterio de continuación** (punto de retorno). Preguntas de diseño: ¿cuáles se pueden manejar y cómo se definen?, ¿cómo se maneja una excepción y cuál es su ámbito?, ¿cómo se lanza?, ¿cómo se especifican los controladores?, ¿a dónde se cede el control al terminar de atenderla?, ¿cómo se propagan?, ¿hay excepciones predefinidas?, ¿hay situaciones no controladas que lleven a mayores fallos? — todo depende de cada lenguaje.

CONTROLADOR DE EXCEPCIONES (MANEJADOR)

Sección de código que se encarga de manejar una excepción en un programa. Su objetivo principal es proporcionar una forma de recuperarse de un error o falla, permitiendo que el programa continúe ejecutándose en lugar de detenerse abruptamente. Según la situación puede: imprimir un mensaje de error, realizar acciones correctivas, lanzar otra excepción, o finalizar la ejecución del programa — debe ser la solución menos perjudicial.

Tipos de excepciones: implícitas (definidas por el lenguaje, built-in) y **explícitas** (definidas por el programador).

PUNTO DE RETORNO Y LOS 2 MODELOS

Después de atender una excepción, el punto de retorno depende del flujo de ejecución, de cómo se diseñó el manejo de excepciones en el código, y del lenguaje. Opciones: continuar la ejecución normal (el punto de retorno es definido por el lenguaje, ej. el bloque siguiente al de manejo); retornar a un estado anterior (deshacer acciones previas a la excepción); propagar la excepción (a un nivel superior en la jerarquía de llamadas, si el controlador local no puede manejarla completamente); o terminar la ejecución del programa (en situaciones críticas, con alguna acción de cierre antes).

- ✓ **Reasunción:** posibilidad de retomar la ejecución normal del programa después de manejar una excepción — el controlador realiza las acciones necesarias (medidas correctivas) y el programa continúa **desde el punto donde se produjo** la excepción. Único lenguaje de este listado que la usa: **PL/1**.
- ✓ **Terminación:** el controlador realiza las acciones necesarias, pero no se retorna al punto donde se produjo la excepción — continúa la ejecución a partir de la **finalización del manejador**. Es el modelo más utilizado actualmente: ADA, CLU, C++, Java, Python, PHP.

PL/I (REASUNCIÓN)

Primer lenguaje que incorporó manejo de excepciones. Las excepciones se llaman **CONDITIONS**. Los manejadores se declaran con `ON CONDITION(Nombre-excepción) Manejador` (el manejador puede ser una instrucción o un bloque `begin/end`). Se lanzan explícitamente con `SIGNAL CONDITION(Nombre-excepción)`.

CONDITION(Nombre-excepcion).

Tiene excepciones predefinidas (built-in) con manejador asociado, ej. `zerodivide` (división por cero). Los manejadores se ligan **dinámicamente** con las excepciones mediante una **pila de manejadores**: una excepción siempre queda ligada al último manejador definido; el alcance de un manejador termina cuando finaliza la ejecución de la unidad donde fue declarado.

ADA (TERMINACIÓN)

Cada vez que se produce una excepción, se termina el bloque donde se levantó y se ejecuta el manejador asociado, y continúa luego. Las excepciones se definen/declaran como variables en la zona de definición (`MiExcepcion: exception;`) y tienen el mismo alcance que la unidad.

La lista de controladores lleva el prefijo `exception`; cada controlador lleva el prefijo `when` seguido de las acciones. `when others` captura cualquier excepción no especificada — debe ir al final del bloque y tiene efectos colaterales (puede tapar excepciones no previstas). Se lanzan explícitamente con `raise`; usar solo `raise` (sin nombre) dentro de un manejador vuelve a levantar la misma excepción que se está manejando, propagándola.

Excepciones predefinidas (built-in)

- ✓ **Constraint_Error:** violar una restricción de una declaración (indexar más allá de los límites de un array, asignar un valor fuera de rango, zerodivide).
- ✓ **Program_Error:** violar la estructura de control o regla del lenguaje (ej. una función termina sin devolver valor).
- ✓ **Storage_Error:** violación de memoria (se requiere más memoria de la disponible).
- ✓ **Tasking_Error:** errores en la comunicación y manejo de tareas del sistema (concurrencia, threads).
- ✓ **Name_Error:** error de nombre (ej. abrir un fichero que no existe).

Propagación

Si la unidad que genera la excepción proporciona un manejador, el control se transfiere de inmediato: se omiten las acciones que siguen al punto donde se generó, se ejecuta el manejador, y el programa continúa desde la instrucción que le sigue. Si no lo proporciona, se termina la unidad (bloque, paquete, subprograma o tarea) y se busca dinámicamente en quien la llamó (se vuelve a levantar en otro ámbito); al propagarse puede volverse **anónima** (la variable excepción ya no está en el alcance) y entrar por `when others`.

La asociación excepción↔manejador es determinística (mismo nombre de variable). Para simular reasunción (continuar ejecutando instrucciones del bloque donde se lanzó), se puede crear un bloque más interno con `Declare` y agregar las instrucciones restantes después de su manejo.

C++ (TERMINACIÓN)

Las excepciones son objetos que pueden ser alcanzados y manejados por manejadores adicionados al bloque donde se produjo la excepción. Cuenta con excepciones estándar predefinidas, organizadas en una jerarquía bajo la clase `exception` (con ramas `logic_error` — domain_error, length_error, out_of_range, invalid_argument — y `runtime_error` — underflow_error, overflow_error, range_error — además de bad_alloc, bad_cast, bad_typeid, etc.).

`try` indica los bloques donde pueden levantarse excepciones; `catch(Tipo)` especifica los manejadores (deben ir después del try y antes de cualquier código externo, asociados a bloques `{ }`); `throw` lanza explícitamente una excepción, y permite pasar parámetros (`throw MiExcepcion(p1,...,pN);`).

Funcionamiento: si se lanza una excepción en el try, el control se transfiere de inmediato al catch cuyo tipo coincide; si se maneja con éxito, la ejecución continúa después del try-catch; si no se encuentra un catch correspondiente, la excepción se propaga hacia bloques try-catch externos (por la jerarquía de llamadas), o termina abruptamente el programa.

Permite especificar en la interfaz de una rutina qué excepciones puede lanzar: `void rutina() throw (Ayuda, Zerodivide);`. Si lanza una excepción no listada, no se propaga: se ejecuta automáticamente `unexpected()` (redefinible), que eventualmente causa `abort()` y termina el programa. Si la lista sí la contempla, se propaga normalmente; si una excepción se propaga repetidamente y nunca encuentra manejador, se llama a `terminate()` (redefinible; por defecto aborta). Sin lista `throw`: cualquier excepción puede propagarse; con lista vacía `throw()`: ninguna puede propagarse.

CLU (TERMINACIÓN)

Solamente pueden ser lanzadas por los **procedimientos**: si una instrucción genera una excepción, el procedimiento que la contiene retorna anormalmente al generarla; no puede manejar una excepción generada por su propia ejecución — quien lo llama debe encargarse. Las excepciones que un procedimiento puede lanzar se declaran en su encabezado (`signals e1, e2`).

Se lanzan explícitamente con `signal`. Los manejadores se colocan al lado de una sentencia (simple o compuesta) con la forma `<instrucción> except <lista_de_controladores> end`, cada rama con `when Nombre-Excepción: Manejador;` y opcionalmente `when others: Manejador;`. Posee excepciones predefinidas con manejador asociado (ej. `failure`) y permite pasar parámetros a los manejadores. Una excepción se puede volver a levantar una sola vez con `resignal`.

Propagación: si no se encuentra el manejador, se termina el procedimiento donde se levantó y se devuelve el control al llamante inmediato, donde debe encontrarse el manejador. Primero se propaga **estáticamente** (dentro de las sentencias incluidas en el mismo procedimiento); si no se encuentra ningún manejador ahí, se levanta una excepción `failure` y devuelve el control terminando todo el programa.

JAVA (TERMINACIÓN)

Igual que C++, las excepciones son objetos manejados por manejadores adicionados al bloque. Cada excepción está representada por una instancia de la clase `Throwable` o de una de sus subclases (`Error` y `Exception`). La gestión usa cinco palabras clave: `try, catch, throw, throws, finally`. Cualquier excepción que se envía desde un método debe especificarse con la cláusula `throws` en su firma. El código que el programador desee que se ejecute siempre (haya o no excepción) va en `finally`.

Fases: detectar e informar el error (lanzamiento con `throw`: un método detecta una condición anormal que le impide continuar y finaliza "lanzando" un objeto Excepción) y recoger el error y tratarlo (captura con bloque `try-catch`: un método recibe el objeto Excepción y decide actuar según el tipo). Orden correcto: `try{...} catch(Tipo1 e){...} catch(Tipo2 e2){...} finally{...}` — finally puede estar o no; si está, se ejecuta cuando termina el try, se haya o no levantado una excepción, salvo que el try haya levantado una que no matcheó con ningún manejador.

PYTHON (TERMINACIÓN)

Se manejan con bloques `try / except`. Primero se ejecuta el bloque try; si no ocurre excepción, el